

BÁO CÁO LAB 3

LẬP BẢN ĐỒ KIẾN TRÚC VÀ LƯỒNG DỮ LIỆU CHO MỘT ỨNG DỤNG WEB ĐƠN GIẢN

Họ tên: MSSV: Lớp:

Hình thức: Cá nhân. Mã nguồn: github.com/UngVangDaiLam/ATw-CSDL_Lab (thư mục lab2/login-app).

1. Ứng dụng được chọn

Ứng dụng **đăng nhập** đã xây dựng ở Lab 2: trang HTML/CSS/JS gửi dữ liệu bằng **fetch (AJAX)** tới server **Node.js + Express**; server kiểm tra tài khoản (admin / 12345), trả về JSON và lưu trạng thái đăng nhập bằng **session cookie**. Vì phân tích trên code thật nên sơ đồ phản ánh đúng các thành phần đang chạy.

2. Các thành phần chính

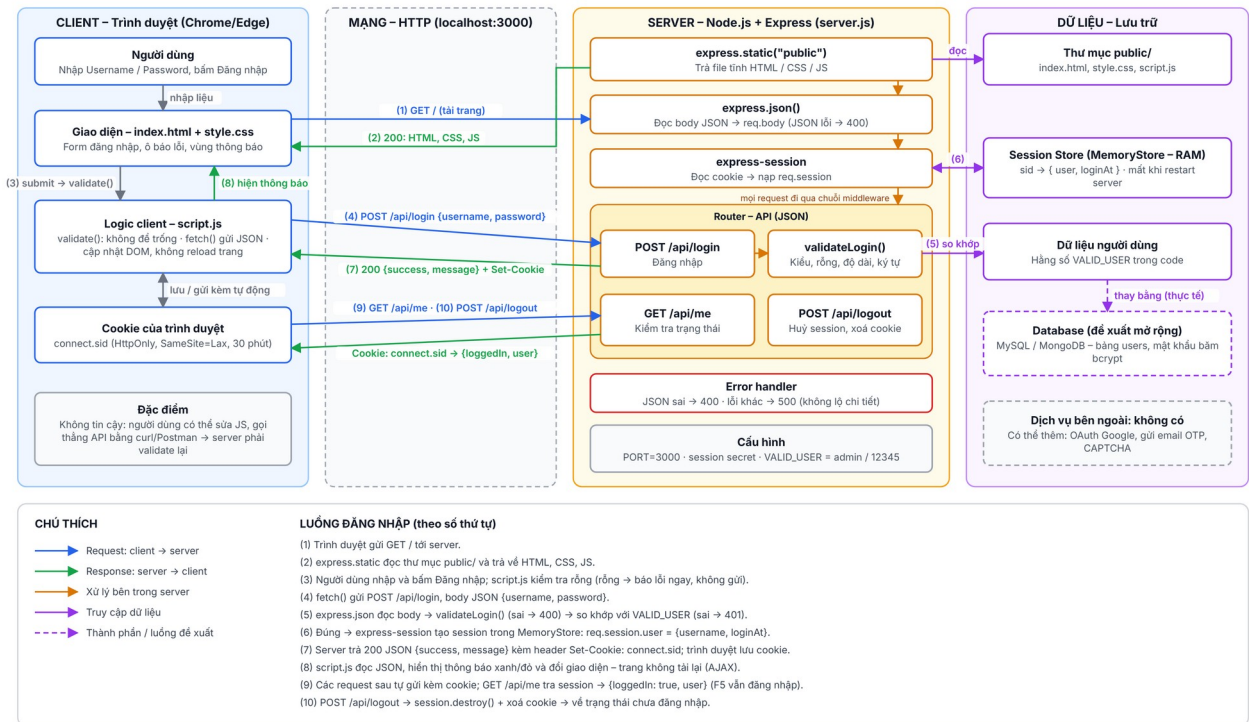
Tầng	Thành phần	Công nghệ	Vai trò
Client	Giao diện (index.html, style.css)	HTML5, CSS3	Hiển thị form, ô báo lỗi, vùng thông báo.
Client	Logic client (script.js)	JavaScript, Fetch API	Kiểm tra rỗng, gửi JSON, cập nhật DOM không reload.
Client	Cookie trình duyệt	connect.sid	Giữ mã phiên; tự gửi kèm mọi request tới server.
Mạng	Kênh giao tiếp	HTTP/1.1, JSON	localhost:3000; chưa mã hoá (chưa dùng HTTPS).
Server	Middleware	express.static, express.json, express-session	Trả file tĩnh, đọc body JSON, nạp session từ cookie.
Server	API (Router)	Express Router	POST /api/login, GET /api/me, POST /api/logout.
Server	validateLogin(), Error handler	JavaScript	Validate lại phía server (400/401); bắt lỗi JSON, trả 500 chung.
Dữ liệu	public/, Session Store, VALID_USER	File tĩnh, MemoryStore (RAM), hằng số	Lưu file giao diện, phiên đăng nhập và tài khoản hợp lệ. Chưa có CSDL.
Bên ngoài	Không có	–	Có thể mở rộng: OAuth Google, email OTP, CAPTCHA.

3. Luồng dữ liệu

- (1) Trình duyệt gửi **GET /**; **express.static** trả về index.html, style.css, script.js.
- (2) Người dùng nhập và bấm Đăng nhập; **script.js** kiểm tra ô trống (trống → báo lỗi ngay, không gửi).
- (3) **fetch()** gửi **POST /api/login**, header Content-Type: application/json, body {username, password}.
- (4) **express.json** đọc body → **validateLogin()** kiểm tra kiểu, độ dài, ký tự (sai → **400**) → so khớp với **VALID_USER** (sai → **401**).
- (5) Đúng → **express-session** tạo phiên trong MemoryStore (req.session.user = {username, loginAt}); server trả **200** JSON {success, message} kèm **Set-Cookie: connect.sid** (HttpOnly).
- (6) script.js đọc JSON và hiển thị thông báo xanh/đỏ ngay trên trang (AJAX, không tải lại).
- (7) Các request sau, trình duyệt tự gửi kèm cookie: **GET /api/me** tra phiên → {loggedIn, user} (F5 vẫn còn đăng nhập); **POST /api/logout** huỷ phiên và xoá cookie.

Sơ đồ chi tiết (Hình 1) và sơ đồ tuần tự có các nhánh lỗi 400/401 nằm trong file **so-do-kien-truc.pdf** (file gốc: so-do-kien-truc.drawio, mở bằng draw.io).

BẢN ĐỒ KIẾN TRÚC & LƯỒNG DỮ LIỆU – ỨNG DỤNG ĐĂNG NHẬP (Lab 2: HTML/CSS/JS + Node.js Express)



Hình 1. Bản đồ kiến trúc và luồng dữ liệu của ứng dụng đăng nhập

4. Phân tích sơ bộ về bảo mật

Điểm đã làm tốt: validate ở cả client và server; cookie **HttpOnly** (JS không đọc được cookie khi bị XSS) và **SameSite=Lax** (giảm CSRF); hiển thị thông báo bằng textContent thay vì innerHTML (hạn chế XSS); thông báo lỗi chung "Sai username hoặc password" không tiết lộ username nào tồn tại; error handler không trả stack trace cho client.

Điểm yếu tiềm năng	Rủi ro	Đề xuất khắc phục
Mật khẩu ghi cứng, dạng rõ trong server.js	Lộ tài khoản khi đưa code lên GitHub công khai	Lưu user trong CSDL, mật khẩu băm bcrypt ; cấu hình qua file .env
Session secret ghi cứng trong code	Kẻ tấn công có thể giả mạo chữ ký cookie	Đọc từ biến môi trường, chuỗi ngẫu nhiên đủ dài
Dùng HTTP, chưa có HTTPS	Nghe lén mật khẩu và cookie trên mạng (MITM)	Bật HTTPS (TLS), đặt cookie.secure = true
Không giới hạn số lần đăng nhập sai; mật khẩu yếu (12345)	Tấn công dò mật khẩu (brute force)	express-rate-limit, khoá tạm thời, CAPTCHA, chính sách mật khẩu mạnh
Không tạo lại session sau khi đăng nhập	Session fixation	Gọi req.session.regenerate() trước khi gán user
Session lưu trong RAM (MemoryStore)	Mất phiên khi restart, rò rỉ bộ nhớ, không chạy được nhiều server	Dùng Redis hoặc CSDL làm session store
Chưa có CSRF token, chưa có security headers	CSRF (trình duyệt cũ), clickjacking	Thêm CSRF token; dùng helmet (CSP, X-Frame-Options...)

5. Kết luận

Ứng dụng có kiến trúc client–server 3 tầng rõ ràng: trình duyệt lo giao diện và kiểm tra sơ bộ, server Express xử lý nghiệp vụ và xác thực, tầng dữ liệu lưu file tĩnh và phiên đăng nhập. Hai bên chỉ giao tiếp qua HTTP + JSON nên dễ thay thế từng phần. Điểm cần cải thiện lớn nhất là thay tài khoản ghi cứng bằng CSDL có mật khẩu băm, chuyển sang HTTPS và giới hạn số lần đăng nhập sai trước khi đưa vào môi trường thật.